Statistical Arbitrage Engine

Pairs Trading with Cointegration, Kalman Filtering & Alpaca Paper Trading

Project Type	Quantitative Finance — Algorithmic Trading
Methods	Engle-Granger, Johansen, Kalman Filter, OLS, ADF
Implementation	Python 3.10+ | NumPy | Pandas | Alpaca API
Trading Mode	Paper Trading (Alpaca) + Full Backtesting
Document	Theory, Architecture, Code Reference, Deployment

1. Introduction to Statistical Arbitrage

1.1 What is Statistical Arbitrage?

Statistical arbitrage (stat arb) is a quantitative trading strategy that exploits temporary mispricings between related securities. Unlike classical arbitrage, which guarantees riskless profit, stat arb relies on the probabilistic tendency of price relationships to revert to a historical mean. The core insight is that while individual stock prices follow near-random walks, certain pairs or portfolios of stocks maintain a stable long-run equilibrium — a property called **cointegration**.

1.2 Core Mechanics

The strategy operates in three phases: (1) identify cointegrated pairs via statistical testing, (2) compute the spread between the pair and monitor its deviation from the mean, and (3) enter a market-neutral position when the spread is sufficiently extreme, then close when it mean-reverts.

Key advantages of pairs trading:

- **Market neutral:** Long one asset, short the other — insulated from broad market moves.
- **Mean-reverting signal:** Predictable return-to-mean behavior gives a statistical edge.
- **Scalable:** Works across equities, ETFs, commodities, FX, and crypto.
- **Low correlation to benchmarks:** Decorrelated alpha source for portfolio construction.

2. Cointegration Theory

2.1 Stationarity and Unit Roots

A time series X_t is **stationary** (I(0)) if its mean, variance, and autocovariance are constant over time. Most financial price series are non-stationary — they are integrated of order one, I(1), meaning their first differences are stationary. A random walk is the canonical example:

```
P_t = P_{t-1} + epsilon_t (epsilon_t ~ N(0, sigma^2))
```

2.2 Definition of Cointegration

Two I(1) series X_t and Y_t are **cointegrated** if there exists a linear combination that is I(0). Formally, there exists beta such that:

```
Z_t = Y_t - beta * X_t - alpha ~ I(0)
```

The series Z_t is called the **spread** or **cointegrating residual**. Its stationarity implies mean reversion — deviations from the equilibrium are temporary and will eventually revert. The parameter beta is the **hedge**

ratio that makes the portfolio dollar-neutral.

2.3 Engle-Granger Two-Step Test

The Engle-Granger (1987) procedure tests for cointegration via two steps:

Step 1 — OLS Regression: Regress Y_t on X_t to obtain the hedge ratio:

```
Y_t = beta * X_t + alpha + u_t [OLS]
```

Step 2 — ADF Test on Residuals: Test whether the estimated residuals u■$_t$ = Y_t - beta■ X_t - alpha■ are stationary using the Augmented Dickey-Fuller test:

```
Delta u_t = gamma * u_{t-1} + sum_i phi_i * Delta u_{t-i} + e_t
```

H_0: gamma = 0 (unit root, no cointegration). H_1: gamma < 0 (stationary, cointegrated). Rejection of H_0 at the 5% level (t-stat < -2.86 using MacKinnon critical values) implies cointegration.

2.4 Johansen Multivariate Test

The Johansen (1991) test overcomes a key limitation of Engle-Granger: it does not assume a priori which variable is on the left-hand side. It tests for the rank r of the cointegration space in a Vector Error Correction Model (VECM):

```
Delta Y_t = Pi * Y_{t-1} + sum_{i=1}^{k-1} Gamma_i * Delta Y_{t-i} +
epsilon_t
```

where Pi = alpha * beta' is decomposed into loading matrix alpha and cointegrating matrix beta. The **trace statistic** tests H_0: rank(Pi) = r vs H_1: rank(Pi) > r. For a bivariate system (p=2), the 5% critical value for r=0 is 15.41.

3. Kalman Filter for Dynamic Hedge Ratios

3.1 The Problem with Static OLS

OLS estimates a single, static hedge ratio beta over the entire history. In practice, the cointegrating relationship drifts over time due to structural changes, regime shifts, and evolving market microstructure. A static beta can produce a spread that is non-stationary over rolling windows, generating false signals.

3.2 State-Space Formulation

The Kalman filter treats the hedge ratio as a latent state variable that evolves over time. We define the state-space model as:

Observation Eq.	y_t = H_t * theta_t + epsilon_t, epsilon_t ~ N(0, R)

State Transition	theta_t = theta_{t-1} + w_t, w_t ~ N(0, Q)
State Vector	theta_t = [beta_t, alpha_t]'
Observation Vec.	H_t = [x_t, 1] (price of asset A and intercept)
Process Noise Q	delta / (1 - delta) * I_2, delta controls beta drift speed
Obs. Noise R	Scalar; represents measurement uncertainty in price B

3.3 Kalman Filter Update Equations

The filter proceeds recursively for each new observation:

```
Innovation: nu_t = y_t - H_t * theta_{t|t-1}
```

```
Innovation Var: S_t = H_t * P_{t|t-1} * H_t' + R
```

```
Kalman Gain: K_t = P_{t|t-1} * H_t' * S_t^{-1}
```

```
State Update: theta_{t|t} = theta_{t|t-1} + K_t * nu_t
```

```
Cov. Update: P_{t|t} = (I - K_t * H_t) * P_{t|t-1} + Q
```

3.4 Parameter Tuning

The delta parameter (process noise) controls the speed of beta adaptation. A smaller delta (e.g., 1e-5) assumes near-constant beta; a larger value (e.g., 1e-3) allows rapid adaptation. In practice, delta is calibrated via maximum likelihood on historical data or set based on expected regime change frequency.

4. Signal Generation and Position Sizing

4.1 Rolling Z-Score

Once the dynamic spread $Z_t = Y_t - \text{beta}_t X_t - \text{alpha}_t$ is computed, we standardize it over a rolling window of length w:

```
z_t = (Z_t - mu_w) / sigma_w
```

where mu_w and sigma_w are the rolling mean and standard deviation of the spread over the past w periods. The z-score measures how many standard deviations the current spread is from its recent mean.

4.2 Entry and Exit Rules

Long Spread Entry	z_t < -theta_entry → Buy A, Sell B (spread too low, expect rise)
Short Spread Entry	z_t > +theta_entry → Sell A, Buy B (spread too high, expect fall)
Exit (mean revert)	|z_t| < theta_exit → Close position (spread returned to mean)
Stop Loss	|z_t| > theta_stop → Close position (spread diverged further)
Typical Values	theta_entry = 2.0, theta_exit = 0.5, theta_stop = 3.5

4.3 Half-Life of Mean Reversion

The half-life measures how quickly the spread reverts to its mean. It is estimated by fitting an AR(1) model to the spread and computing:

```
Delta Z_t = theta * (mu - Z_{t-1}) + epsilon_t
```

```
Half-Life = log(2) / theta
```

Pairs with very short half-lives (<5 days) may be driven by microstructure noise; pairs with very long half-lives (>60 days) revert too slowly to trade profitably given transaction costs. The sweet spot is typically 10–30 trading days.

4.4 Dollar-Neutral Position Sizing

To maintain market neutrality, position sizes are set so that the dollar value of the long leg equals the dollar value of the short leg, scaled by the hedge ratio:

```
qty_B = floor(Capital / Price_B)
```

```
qty_A = floor(qty_B * |beta|)
```

5. Backtesting Methodology

5.1 Walk-Forward Validation

To avoid overfitting, all backtests use walk-forward validation: the model is trained on a historical window and evaluated on the immediately following out-of-sample period. Parameters are re-estimated at each step using only past data.

Training Window	126 trading days (~6 months)

Test Window	21 trading days (~1 month)
Step Size	21 days (non-overlapping test periods)
Cointegration Re-test	At each training step to detect regime changes

5.2 Performance Metrics

Sharpe Ratio	Annualized: (mean_trade_pnl / std_trade_pnl) * sqrt(252)
Win Rate	Number of profitable trades / total trades
Profit Factor	|sum of winning P&L;| / |sum of losing P&L;|
Max Drawdown	max(cumulative_pnl[0:t] - cumulative_pnl[t]) over all t
Half-Life	AR(1) estimate; confirms mean reversion speed
Trade Duration	Average bars between entry and exit

5.3 Common Pitfalls and Mitigations

- **Look-ahead bias:** Ensure all model parameters are estimated using only past data at each signal generation step.
- **Survivorship bias:** Include delisted stocks in the universe; use point-in-time data sources.
- **Transaction costs:** Account for bid-ask spread (typically 1-5bps for liquid equities) and commission per trade.
- **Slippage:** Market impact is material for large positions; model with linear or square-root impact models.
- **Regime changes:** Cointegration can break down. Re-test pairs on a rolling basis (monthly) and exit if cointegration is lost.
- **Multiple testing:** Testing hundreds of pairs inflates false discovery rate. Apply Bonferroni correction or FDR control.

6. Alpaca Paper Trading Integration

6.1 Overview

Alpaca Markets provides a commission-free brokerage API with full paper trading support. The engine connects to Alpaca's REST API at paper-api.alpaca.markets to submit, track, and close positions in real time.

6.2 Key API Endpoints

POST /v2/orders	Submit market, limit, or bracket orders
GET /v2/positions	Retrieve all open positions and unrealized P&L;
DELETE /v2/positions/{symbol}	Close a specific position at market
GET /v2/account	Account equity, buying power, and margin status
GET /v2/trades/{symbol}/latest	Latest trade price for a ticker

6.3 Execution Logic

```
# Long Spread: buy A (undervalued leg), sell B (overvalued leg)
api.submit_order(ticker_a, qty_a, 'buy', 'market', 'gtc')
api.submit_order(ticker_b, qty_b, 'sell', 'market', 'gtc')

# Short Spread: sell A, buy B
api.submit_order(ticker_a, qty_a, 'sell', 'market', 'gtc')
api.submit_order(ticker_b, qty_b, 'buy', 'market', 'gtc')

# Close position on mean reversion or stop loss
api.close_position(ticker_a)
api.close_position(ticker_b)
```

7. System Architecture

7.1 Module Structure

stat_arb_engine.py	Core engine: cointegration tests, Kalman filter, backtester, signal generator, Alpaca executor
live_trader.py	Live trading loop: pair scanner, price polling, signal dispatch
backtest_analysis.py	Walk-forward validation, sensitivity analysis, Kalman vs OLS comparison
stat_arb_engine.jsx	Interactive dashboard: real-time z-score, spread charts, backtest UI, Alpaca order book
requirements.txt	Python dependency list for the full project

7.2 Data Flow

```
Price Feed (yfinance / Alpaca WebSocket)
|
```

```
v
CointegrationTests.engle_granger() + johansen()
| (approve pairs passing both tests)
v
KalmanHedgeFilter.update(price_a, price_b)
| --> beta_t, alpha_t
v
SpreadCalculator.compute() --> spread_t
|
SpreadCalculator.rolling_zscore() --> z_t
|
SignalGenerator.on_bar() --> TradeSignal or None
|
AlpacaExecutor.execute_signal() --> Order submitted
```

8. Parameters Reference

entry_threshold	Z-score to enter a trade. Default: 2.0. Higher = fewer trades, larger edge per trade.
exit_threshold	Z-score to close on reversion. Default: 0.5. Lower = tighter exit, less mean-reversion captured.
stop_loss	Z-score stop loss. Default: 3.5. Prevents holding through cointegration breakdown.
lookback	Rolling window for z-score normalization. Default: 60 bars (~3 months).
capital	Dollar capital per pair. Used for position sizing. Default: $10,000.
delta (Kalman)	Process noise for Kalman filter. Default: 1e-4. Range: 1e-6 (slow) to 1e-2 (fast).
obs_noise (Kalman)	Observation noise R. Default: 1e-3. Larger = smoother beta estimates.
min_half_life	Minimum half-life in days for pair approval. Default: 5 days.
max_half_life	Maximum half-life in days for pair approval. Default: 60 days.

9. Setup and Deployment

9.1 Installation

```
# Clone or download project files
cd stat_arb/

# Install dependencies
pip install -r requirements.txt

# Run the demo (no API keys needed)
python stat_arb_engine.py

# Run full backtest analysis
python backtest_analysis.py
```

9.2 Live Paper Trading

```
# 1. Create free account at alpaca.markets
# 2. Generate Paper Trading API keys
# 3. Export credentials as environment variables

export ALPACA_KEY='PKxxxxxxxxxxxxxxxx'
export ALPACA_SECRET='xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'

# 4. Start the live trader
python live_trader.py
```

10. References

[1] Engle, R.F. and Granger, C.W.J. (1987). Co-integration and Error Correction: Representation, Estimation, and Testing. Econometrica, 55(2), 251-276.

[2] Johansen, S. (1991). Estimation and Hypothesis Testing of Cointegration Vectors in Gaussian Vector Autoregressive Models. Econometrica, 59(6), 1551-1580.

[3] Kalman, R.E. (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering, 82(1), 35-45.

[4] Vidyamurthy, G. (2004). Pairs Trading: Quantitative Methods and Analysis. Wiley Finance.

[5] Chan, E. (2013). Algorithmic Trading: Winning Strategies and Their Rationale. Wiley.

[6] Avellaneda, M. and Lee, J.H. (2010). Statistical Arbitrage in the U.S. Equities Market. Quantitative Finance, 10(7), 761-782.

[7] Hamilton, J.D. (1994). Time Series Analysis. Princeton University Press.

[8] Alpaca Markets API Documentation. https://docs.alpaca.markets